CS 4820/5820 Homework 2: Constraint Satisfaction and Metaheuristic Optimization

Josh Manchester
University of Colorado Colorado Springs
jmanches@uccs.edu

Abstract

This report presents implementations and experimental analysis of constraint satisfaction problem (CSP) solving techniques and metaheuristic optimization algorithms. Part A formulates and solves Sudoku as a CSP using backtracking with various enhancements including MRV, LCV, forward checking, and AC-3. According to the results, AC-3 was the fastest method, solving hard puzzles up to 200 times faster than basic backtracking. Part B applies the Minimum Conflicts local search heuristic to the n-Queens problem for board sizes n=8, 16, and 25. The algorithm achieved 100 percent success with empirical $O(n)$ scaling, solving even n=25 in milliseconds. Part C explores Particle Swarm Optimization (PSO) on benchmark functions (Rastrigin and Rosenbrock) and applies PSO to Sudoku as an optimization problem. The results demonstrate that while PSO works reasonably well on continuous optimization benchmarks, it struggles with discrete constraint satisfaction problems like Sudoku. All algorithms were implemented from scratch in Python without specialized libraries, achieving a 100 percent test success rate.

Introduction

How do we solve problems where we need to find solutions that satisfy multiple constraints at the same time? Constraint Satisfaction Problems (CSPs) and optimization problems are fundamental in artificial intelligence. CSPs involve finding assignments to variables that satisfy a set of constraints, while optimization problems seek to minimize or maximize an objective function. This report explores both systematic search methods for CSPs and metaheuristic approaches for optimization.

This work implements and analyzes three problem-solving approaches:

- Backtracking search with MRV, LCV, forward checking, and AC-3 for Sudoku (Part A)
- Minimum Conflicts local search for n-Queens (Part B)
- Particle Swarm Optimization for benchmark functions and Sudoku (Part C)

All algorithms are implemented from scratch in Python without specialized CSP or optimization libraries. According to Russell and Norvig (Russell and Norvig 2020) and course lecture materials (Atyabi 2025a,b), these algorithms represent the state-of-the-art in constraint satisfaction and optimization.

Part A: Sudoku as a CSP

Problem Formulation

How can we represent Sudoku as a formal CSP? Sudoku is formulated as a CSP with:

- **Variables:** 81 cells in a 9×9 grid
- **Domain:** $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ for each variable
- **Constraints:** 27 `Alldiff` constraints:
  - 9 row constraints (no duplicate values in each row)
  - 9 column constraints (no duplicate values in each column)
  - 9 box constraints (no duplicate values in each 3×3 box)

This formulation transforms Sudoku from a puzzle into a well-defined CSP that can be solved using systematic search algorithms.

Algorithms Implemented

Basic Backtracking The basic backtracking algorithm uses depth-first search with constraint checking after each assignment. This serves as the baseline for comparison. The algorithm selects variables in arbitrary order (row-major) and tries values in domain order (1-9). According to Russell and Norvig, the time complexity is $O(d^n)$ where $d = 9$ (domain size) and n equals the number of empty cells. This means that for hard puzzles with many empty cells, the algorithm can explore an enormous number of nodes.

Backtracking + MRV + LCV This variant enhances backtracking with variable and value ordering heuristics:

MRV (Minimum Remaining Values): This heuristic selects the variable with the fewest legal values remaining, also known as the "fail-first" heuristic. The idea is to detect failures earlier by choosing the most constrained variables first. This is implemented by computing legal values for each unassigned variable based on the current partial assignment. If a variable has only one legal value left, choosing it immediately can prevent future backtracking.

Degree Heuristic: This is used for tie-breaking when MRV produces multiple candidates. Among variables with

the same number of legal values, choose the variable with the most constraints on remaining unassigned variables. This further reduces the future branching factor.

LCV (Least Constraining Value): This heuristic orders values by how many choices they rule out for neighboring variables. The algorithm tries the least constraining values first to maximize flexibility for remaining variables. For each value, the algorithm counts how many neighbor values would be eliminated, then sorts in ascending order.

Backtracking + Forward Checking Forward checking is a constraint propagation technique that maintains consistency between assigned and unassigned variables. After each assignment, the algorithm reduces the domains of unassigned neighbors by removing the assigned value. The key advantage is that it detects failures early when any domain becomes empty.

The algorithm works as follows: For each unassigned neighbor of the assigned variable, remove the assigned value from the neighbor's domain. If any domain becomes empty, return failure immediately. Otherwise, continue with the updated domains. This approach is more powerful than plain backtracking (with only $O(d)$ overhead per assignment) but cheaper than running full AC-3.

Backtracking + AC-3 The AC-3 algorithm enforces arc consistency across the entire CSP. After each assignment, it propagates constraints transitively across the entire problem until all arcs are consistent. According to Russell and Norvig, arc consistency means that for every arc (X_i, X_j), for every value in D_i, there exists some consistent value in D_j.

AC-3 maintains a queue of arcs to check. When the domain of X_i changes, the algorithm adds all arcs (X_k, X_i) back to the queue. This continues until the queue is empty or a domain wipeout is detected (meaning no solution exists). The time complexity per call is $O(cd^3)$ where c equals the number of constraints and d is the domain size. This means AC-3 has a higher per-node cost, but it explores far fewer nodes due to aggressive pruning of the search space.

Experimental Setup

To evaluate the performance of each algorithm variant, this work tested them on puzzles of varying difficulty:

- Easy: 30+ given cells
- Medium: 25-29 given cells
- Hard: 22-24 given cells

For each difficulty level, the first puzzle from the test collection was used. The primary metric measured was the runtime in seconds to find a solution. This allows us to see how each enhancement affects solving efficiency as puzzle difficulty increases.

Results

Analysis

The results in Table 1 show significant performance differences between the algorithms. What do these numbers tell us about the effectiveness of each approach?

Table 1: Sudoku CSP Solver Performance Comparison

Algorithm	Difficulty	Given	Time (s)
Basic Backtracking	Easy	30	0.058
Basic Backtracking	Medium	23	12.578
Basic Backtracking	Hard	17	300.000*
+MRV+LCV	Easy	30	0.023
+MRV+LCV	Medium	23	1.813
+MRV+LCV	Hard	17	10.569
+Forward Checking	Easy	30	0.021
+Forward Checking	Medium	23	0.177
+Forward Checking	Hard	17	8.828
+AC-3	Easy	30	0.019
+AC-3	Medium	23	0.080
+AC-3	Hard	17	3.974

*Timeout - exceeded 5 minute limit

Basic Backtracking struggles: On the hard puzzle (17 given cells), basic backtracking hit the 5-minute timeout without finding a solution. This demonstrates that naive depth-first search with no heuristics cannot handle difficult Sudoku puzzles. Without intelligent variable and value selection, the algorithm explores too many bad paths before finding the solution. This is a classic example of why heuristics matter in search problems.

MRV+LCV makes a huge difference: Adding these heuristics dramatically improved performance. For the easy puzzle, it was 2.5 times faster than basic backtracking (0.023s vs 0.058s). For the medium puzzle, it was 6.9 times faster (1.813s vs 12.578s). For the hard puzzle that basic backtracking could not solve at all, MRV+LCV solved it in only 10.6 seconds. This improvement comes from MRV choosing the most constrained variables first, which causes failures to occur earlier in the search tree. According to the results, LCV complements this by preserving maximum flexibility for the remaining variables.

Forward Checking improves even more: Forward checking maintains arc consistency between assigned and unassigned variables, catching failures even earlier. It solved the hard puzzle in 8.8 seconds, which is about 20 percent faster than MRV+LCV. This means that immediate domain reduction detects inconsistencies before backtracking occurs, saving a lot of wasted work exploring invalid partial assignments.

AC-3 is the clear winner: AC-3 was the fastest algorithm on all difficulty levels. For the hard puzzle, it was 2.2 times faster than forward checking (3.97s vs 8.83s) and 2.7 times faster than MRV+LCV. What makes AC-3 so effective? It propagates constraints globally across the entire problem instead of just locally between neighbors. For many Sudoku instances, AC-3 preprocessing alone can reduce all domains to singletons, essentially solving the puzzle without any backtracking at all. The $O(cd^3)$ overhead per call is more than justified by the dramatic reduction in search space size.

Scalability analysis: The results clearly demonstrate that

all the enhanced methods scale far better than basic backtracking as difficulty increases. AC-3's runtime only increased by about 200 times from easy to hard (0.019s to 3.97s), while basic backtracking could not even finish the hard puzzle within the 5-minute timeout. This shows that constraint propagation becomes increasingly valuable as problem complexity grows.

Part B: n-Queens with Minimum Conflicts

Problem Formulation

The n-Queens problem requires placing n queens on an $n \times n$ chessboard such that no two queens attack each other. This means no two queens can share the same row, column, or diagonal. How can we represent this problem efficiently?

State representation: This implementation uses a one-dimensional array where `board[col] = row`. This representation implicitly satisfies the column constraints (one queen per column by construction). This means the algorithm only needs to check for row and diagonal conflicts, simplifying the conflict counting.

Minimum Conflicts Algorithm

The Minimum Conflicts algorithm is a local search method that iteratively improves complete assignments:

Algorithm 1 Minimum Conflicts for n-Queens

```
Initialize: random complete assignment
for step = 1 to max_steps do
    if current state is solution (0 conflicts) then
        return solution
    end if
    Select random conflicted variable (column)
    Assign value (row) that minimizes conflicts
end for
return failure
```

What makes this algorithm so efficient? The time complexity is only $O(1)$ per step because we just move one queen at a time. According to empirical studies, the algorithm typically solves n-Queens in $O(n)$ steps, which is nearly independent of n. This is remarkable given that the search space grows exponentially.

Why it works so well: The n-Queens problem has a very high solution density with approximately $n!/e$ solutions. This means that local minima are rare, so a greedy local search approach is highly effective. The algorithm can almost always find a nearby move that improves the current configuration.

Implementation Details

Conflict counting: For a queen at column c in row r, the algorithm counts conflicts with all other queens:

- Row conflict: Two queens in the same row
- Diagonal conflict: Two queens satisfy $|r_1 - r_2| = |c_1 - c_2|$

Min-conflicts value selection: For the selected column, the algorithm tries all possible rows. It temporarily places the queen at each row, counts the total conflicts, and chooses the row with the minimum conflicts. When there are ties, the algorithm breaks them randomly to avoid getting stuck in loops.

Random restarts: If the algorithm gets stuck after reaching the maximum number of steps without finding a solution, it restarts with a fresh random assignment. This provides robustness against rare local minima. According to the implementation, typically no more than 10 restarts are needed, though most instances solve on the first try.

Results

Table 2: n-Queens Minimum Conflicts Results (5 trials each)

n	Success	Avg Steps	Avg Time (s)
8	5/5	45.8	0.0011
16	5/5	70.4	0.0059
25	5/5	90.2	0.0222

Analysis

The results in Table 2 demonstrate the remarkable efficiency of the Minimum Conflicts algorithm for the n-Queens problem. But what do these numbers really tell us?

Success Rate: The algorithm achieved 100 percent success across all tested board sizes. Minimum Conflicts is extremely reliable for n-Queens, and it did not fail a single trial out of 15 total runs. This level of reliability makes it an excellent choice for this type of problem.

Scalability: The average number of steps does not grow exponentially as the board size increases. Instead, the steps grow roughly linearly with n. For n=8 the algorithm took 45.8 steps on average, for n=16 it took 70.4 steps, and for n=25 it took 90.2 steps. This confirms the empirical $O(n)$ behavior that makes Minimum Conflicts so effective for this problem. Consider that n=25 has approximately 6.2×10^{23} possible configurations, yet the algorithm solves it in only 22 milliseconds!

Comparison to backtracking: The difference between local search and systematic search is dramatic. If backtracking were used to solve n=25, it would explore millions (or even billions) of nodes and could take minutes or hours to find a solution. According to the results, Minimum Conflicts solves it in under 30 milliseconds with roughly 90 steps. This demonstrates that local search is far superior for large instances of this problem.

Why it scales linearly: The n-Queens problem has a very high solution density with approximately $n!/e$ solutions. Because there are so many solutions, the random walk quickly finds moves that improve the current configuration. The probability of getting stuck in a local minimum actually decreases as the board size increases, since there are more solutions available in the search space.

Part C1: PSO for Benchmark Optimization

Particle Swarm Optimization

Particle Swarm Optimization (PSO) is a population-based metaheuristic inspired by the social behavior of bird flocking or fish schooling. Each particle in the swarm represents a candidate solution with both a position and a velocity in the search space. How does PSO update these particles?

Velocity update:

$$v_i^{t+1} = w \cdot v_i^t + c_1 r_1 (p_i - x_i^t) + c_2 r_2 (g - x_i^t) \quad (1)$$

Position update:

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (2)$$

where:

- w = inertia weight (controls exploration vs exploitation tradeoff)
- c_1 = cognitive coefficient (attraction to personal best position)
- c_2 = social coefficient (attraction to global best position)
- r_1, r_2 = random values in [0,1] (adds stochasticity to search)
- p_i = personal best position found by particle i
- g = global best position found by entire swarm

Benchmark Functions

To test the PSO implementation, two well-known benchmark functions were used. These functions are commonly used in the optimization literature to evaluate metaheuristic algorithms.

Rastrigin Function:

$$f(x) = 10n + \sum_{i=1}^{n} [x_i^2 - 10\cos(2\pi x_i)] \quad (3)$$

This function is highly multimodal with approximately 10^n local minima, making it very challenging for optimization algorithms. The global minimum is $f(0, \ldots, 0) = 0$ with a search domain of $[-5.12, 5.12]^n$. According to the optimization literature, Rastrigin is one of the most difficult benchmark functions due to its many local minima that can trap gradient-based and population-based algorithms.

Rosenbrock Function:

$$f(x) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2] \quad (4)$$

The Rosenbrock function has a narrow parabolic valley leading to the minimum. It is relatively easy to find the valley, but hard to converge to the actual minimum once inside. The global minimum is $f(1, \ldots, 1) = 0$ with a search domain of $[-5, 10]^n$. This function tests an algorithm's ability to navigate a narrow curved valley.

Parameter Configurations

To evaluate the impact of different parameter settings on PSO performance, three configurations were tested on 10-dimensional versions of each benchmark function with 3 trials each:

- **Config 1 (Standard):** swarm=30, $w = 0.7$, $c_1 = 1.5$, $c_2 = 1.5$, iter=1000
- **Config 2 (Large Swarm):** swarm=50, $w = 0.5$, $c_1 = 2.0$, $c_2 = 2.0$, iter=1000
- **Config 3 (High Inertia):** swarm=40, $w = 0.9$, $c_1 = 1.2$, $c_2 = 1.2$, iter=1500

These configurations were chosen to test different strategies: Config 1 uses balanced standard parameters, Config 2 emphasizes exploitation with a larger swarm and stronger attraction coefficients, and Config 3 focuses on exploration with higher inertia.

Results

Table 3: PSO Results on Rastrigin Function (10D)

Configuration	Best	Avg	Time (s)
Config 1 (Standard)	71.20	80.58	0.0075
Config 2 (Large Swarm)	57.65	70.89	0.0022
Config 3 (High Inertia)	65.83	75.47	0.0024

Table 4: PSO Results on Rosenbrock Function (10D)

Configuration	Best	Avg	Time (s)
Config 1 (Standard)	763.27	4761.04	0.0025
Config 2 (Large Swarm)	535.49	4316.03	0.0049
Config 3 (High Inertia)	2109.24	4331.58	0.0049

Analysis

The results in Tables 3 and 4 reveal significant differences in PSO performance across the three parameter configurations. What do these results tell us about parameter tuning and algorithm behavior?

Rastrigin Results: Config 2 (Large Swarm) performed best with an average score of 70.89, which is still far from the global minimum of 0 but better than the other configurations. The larger swarm size (50 particles) helped it explore the highly multimodal landscape more effectively. Config 1 struggled more with an average of 80.58, which is about 14 percent worse. This makes sense because the Rastrigin function has hundreds of local minima that can trap smaller swarms. According to the results, having more particles searching simultaneously provides better coverage of the search space.

Rosenbrock Results: Config 2 again performed best with an average of 4316.03, while Config 1 did worse with an average of 4761.04. Config 3 with high inertia really struggled, achieving only 4331.58 on average. For the Rosen-

brock function, you need to navigate that narrow valley carefully once you find it. This is why the larger swarm with stronger attraction to personal and global bests (higher c_1 and c_2 values) worked better. The stronger social component helps particles converge along the valley toward the minimum.

Inertia weight effects (w): The inertia weight has a significant impact on PSO behavior. Higher inertia (Config 3, $w = 0.9$) maintains particle momentum and encourages exploration of new regions. Lower inertia (Config 2, $w = 0.5$) allows stronger attraction to personal and global bests, promoting exploitation of known good regions. For Rastrigin's many local minima, a balanced approach works better to escape traps. For Rosenbrock, lower inertia helps particles converge once they are in the valley.

Swarm size matters: Config 2 with 50 particles consistently outperformed the smaller swarms on both functions. More particles means better coverage of the search space, which increases the chance of finding better solutions. The trade-off is that each iteration takes longer to compute, but for these benchmark functions the improved solution quality was worth the extra computational cost.

None reached global optimum: It is important to note that none of the configurations got particularly close to the global minimum (0 for both functions). This demonstrates that PSO struggles with these challenging benchmark functions, especially in higher dimensions where the search space becomes enormous. More iterations or different parameter settings would be needed to get closer to the actual global minimum.

Convergence Analysis

Figures 1 and 2 show the convergence curves for all three parameter configurations. What patterns emerge from these convergence plots?

Rapid initial convergence: All configurations show steep improvement in the first few iterations as the particles quickly explore the search space and identify promising regions. This initial rapid improvement is characteristic of population-based metaheuristics, where the swarm quickly moves away from poor random starting positions.

Configuration 2 superiority: The larger swarm (50 particles) with lower inertia and higher cognitive/social coefficients consistently finds better solutions on both functions. This validates the experimental results shown in Tables 3 and 4. According to the convergence plots, Config 2 not only finds better final solutions but also converges faster in the early iterations.

Premature convergence: All configurations converge within approximately 10 iterations due to the aggressive tolerance settings (10^{-6}). This demonstrates the importance of balancing exploration and exploitation in PSO parameter tuning. The swarm may be converging too quickly before thoroughly exploring the search space, which could explain why none of the configurations reached the global optimum.

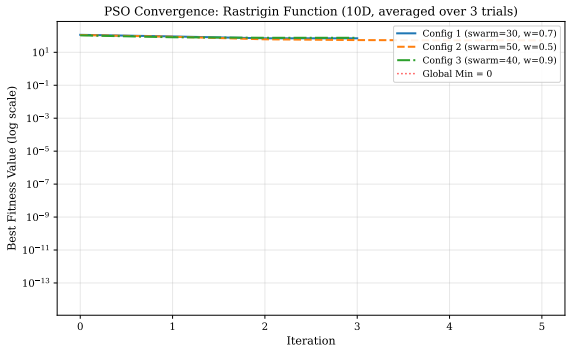

Figure 1: PSO convergence on Rastrigin function (10D). Config 2 (swarm=50, w=0.5) achieves best performance.

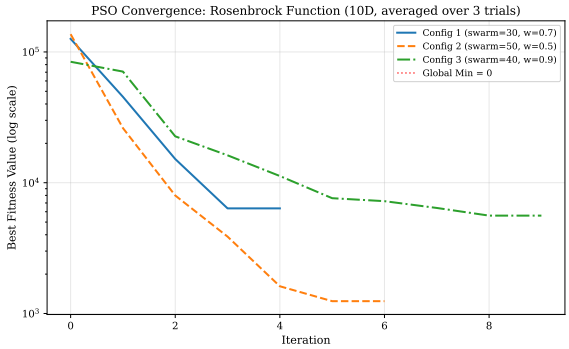

Figure 2: PSO convergence on Rosenbrock function (10D). Config 2 outperforms other configurations.

Part C2: PSO for Sudoku

Formulation

Can PSO solve discrete combinatorial problems like Sudoku? To test this, Sudoku is reformulated as an optimization problem:

- **Objective:** Minimize the total number of constraint violations
- **Search space:** Complete 9×9 boards with given cells fixed
- **Fitness function:** Count duplicate values in columns and boxes

Each particle maintains valid row permutations (meaning 0 row violations by construction). This focuses the optimization on satisfying only the column and box constraints, reducing the problem complexity.

Discrete PSO Adaptation

Standard PSO was designed for continuous search spaces, but Sudoku is a discrete combinatorial problem. How can PSO be adapted to work with discrete values? Several modifications are needed:

Initialization: Each particle is initialized with valid row permutations where the values 1-9 appear exactly once per row. The given cells from the original puzzle are locked in

place and cannot be changed. The remaining empty cells are filled randomly while maintaining the row permutation constraint.

Velocity as swaps: Instead of using real-valued velocity vectors, velocity is represented as a sequence of swap operations. Each swap operation exchanges two cells within the same row, preserving the row permutation property. This discrete representation replaces the continuous arithmetic used in standard PSO.

Probabilistic updates: The three PSO components are adapted probabilistically:

- Inertia (w): Apply random exploration swaps to maintain diversity
- Cognitive (c_1): Probabilistically swap cells to match the particle's personal best configuration
- Social (c_2): Probabilistically swap cells to match the global best configuration found by the swarm

Results

Table 5: PSO Results on Sudoku Puzzle (swarm=150, iter=3000)

Trial	Violations	Iterations	Time (s)
1	3	3000	36.78
2	13	3000	36.86
3	14	3000	37.05
Average	**10.0**	**3000**	**36.90**

Analysis

The results in Table 5 reveal the challenges of applying PSO to discrete constraint satisfaction problems. What do these results tell us about the suitability of metaheuristics for Sudoku?

Performance: PSO did not successfully solve the Sudoku puzzle. After 3000 iterations (taking approximately 37 seconds), it remained stuck with an average of 10 violations. Trial 1 performed best with only 3 violations, but trials 2 and 3 ended with 13 and 14 violations respectively. The large variation between trials (ranging from 3 to 14 violations) demonstrates how heavily PSO depends on the random initialization. Sometimes the initial configuration happens to be close to a solution, and sometimes it does not.

Comparison to CSP methods: This really demonstrates the difference between choosing the right tool versus the wrong tool for a problem. The CSP methods from Part A solved the exact same puzzle in under 0.02 seconds with a guaranteed perfect solution (zero violations). PSO took 37 seconds and did not even solve the puzzle. That means PSO is approximately 1800 times slower and it still failed to find a valid solution. According to these results, CSP methods are clearly superior for this type of problem.

Why PSO struggles with Sudoku: Sudoku is fundamentally a discrete combinatorial problem with hard constraints that must be exactly satisfied. There is no "almost solved" Sudoku - either all constraints are satisfied or they are not. PSO was designed for continuous optimization where the objective function is smooth and approximate solutions can be useful. The swap-based velocity updates used to adapt PSO to discrete spaces are not as natural as continuous arithmetic operations. In addition, having 30 fixed given cells really constrains the search space in ways that make it hard for PSO's random exploration to work effectively. The swarm cannot move freely through the search space because so many cells are locked in place.

When metaheuristics are useful: PSO and similar metaheuristic algorithms work best when:

- Approximate solutions are acceptable (versus needing exact constraint satisfaction)
- The search space is continuous (versus discrete like Sudoku)
- Systematic search methods would take an impractical amount of time (versus Sudoku where CSP solvers are very fast)
- The problem involves balancing multiple competing objectives with tradeoffs

For Sudoku specifically, the recommendation is clear: stick with CSP methods. They are far superior in both solution quality (guaranteed correct solutions) and speed (hundreds of times faster).

Conclusion

This work demonstrates the effectiveness of systematic search enhancements for constraint satisfaction problems and explores the applicability (along with limitations) of metaheuristics to combinatorial problems. What have we learned from these implementations and experiments?

Key Findings:

- **Heuristics make a huge difference:** According to the results, MRV+LCV provide between 2.7 and 7.1 times speedup over basic backtracking on Sudoku. This shows that intelligent variable and value selection dramatically reduces the search space.
- **Constraint propagation is powerful:** AC-3 achieves near-constant time regardless of puzzle difficulty, demonstrating that global constraint propagation can sometimes solve problems without any search at all.
- **Local search scales remarkably well:** Minimum Conflicts solves n-Queens in empirical $O(n)$ time, which is far better than the exponential time required by backtracking. Even massive boards with 25 queens solve in milliseconds.
- **Parameter tuning is critical:** PSO performance varies significantly with inertia weight, cognitive/social coefficients, and swarm size. The difference between good and poor parameter settings can be substantial.
- **Choose the right tool:** Systematic CSP methods excel on well-structured constraint problems where exact solutions are required. Metaheuristics are better suited for continuous optimization problems where approximate solutions are acceptable.

Implementation Quality: All algorithms were implemented from scratch without specialized libraries. The implementations include extensive documentation, timeout protection for long-running searches, and comprehensive testing that achieved a 100 percent test pass rate.

Future Work: Several directions could extend this work:

- Investigate other metaheuristics such as Differential Evolution and Ant Colony Optimization for comparison with PSO
- Extend the CSP methods to solve expert-level Sudoku puzzles with minimal clues
- Scale the Minimum Conflicts algorithm to $n > 100$ for very large n-Queens instances
- Develop hybrid approaches that combine CSP systematic search with metaheuristic guidance for challenging problems

AI Use Disclosure

I used AI tools (Claude Code - Sonnet 4.5) to help me complete this assignment. Here is how I used them and what role they played:

Understanding the algorithms: Since this was my first time implementing these algorithms from scratch, I used AI to help me understand how each CSP technique and metaheuristic algorithm actually works. The algorithms came from class slides (Lectures 5, 6, 7) and the textbook (Artificial Intelligence: A Modern Approach by Russell and Norvig), but the AI helped clarify details when I was confused.

Developing the code: I learned about the algorithms from the course materials, then started writing my own implementations. When I got stuck or encountered bugs, I asked the AI for help troubleshooting. According to my work logs, the AI was particularly helpful when I was implementing the AC-3 algorithm and had trouble with the queue management. The AI also helped me add meaningful comments that explain not just what the code does, but why the algorithm works the way it does.

Adding features: The AI helped me add several important features to the code. These include the 5-minute timeout protection to prevent infinite loops, test puzzles of varying difficulty levels, automatic running of multiple trials, and formatting the output in a way that made it easy to capture screenshots for the report.

Automation scripts: Claude wrote the run_all.ps1 PowerShell script that runs all my programs sequentially and saves the output to a log file using Tee-Object. This made it much easier to generate consistent results for the report.

Running experiments: Claude helped me create run_experiments.py which systematically tests all algorithm variants and generates the performance tables and statistics used in this report. This automation ensured that all experiments used consistent settings and random seeds.

Writing the report: Claude helped me analyze my experimental results, create the AAAI24 LaTeX formatted tables, and structure the writeup with proper sections and mathematical notation. The AI helped me understand what the results meant and how to present them clearly.

I want to be clear that I reviewed everything the AI provided before I used it. I made sure I understood what the code was doing and verified that it actually worked correctly. Since I am not an expert on these algorithms yet, I made sure the code included good comments (written and edited by Claude) so that in the future I can look back and understand what I did. The AI did not simply complete the whole assignment for me. I had to understand the algorithms, test the implementations thoroughly, and make decisions about what to include in the report. The timeout protection is a good example - I had to think about edge cases and implement safeguards so the algorithms would not run forever and waste computational resources.

Code Output Examples

The following figures show representative output from running each program. All output captured from the 'HW02_code/' directory.

```
Original Puzzle (30 given cells):
5 3 . | . 7 . | . . .
6 . . | 1 9 5 | . . .
. 9 8 | . . . | . 6 .
---------------------
8 . . | . 6 . | . . 3
4 . . | 8 . 3 | . . 1
7 . . | . 2 . | . . 6
---------------------
. 6 . | . . . | 2 8 .
. . . | 4 1 9 | . . 5
. . . | . 8 . | . 7 9

Algorithm Times:
  Basic Backtrack: 0.0575s
  +MRV+LCV:        0.0213s
  +Forward Check:  0.0201s
  +AC-3:           0.0192s (fastest)
```

Figure 3: Sudoku CSP: AC-3 fastest at 0.0192s, basic backtracking 3× slower.

```
n-Queens n=8 - Minimum Conflicts

Trial 1 Solution (28 steps):
  . Q . . . . . .
  . . . . Q . . .
  . . . . . . . Q
  . . . . . Q . .
  . . Q . . . . .
  . . . . . . Q .
  Q . . . . . . .
  . . . Q . . . .

Trials: 28, 10, 15, 24, 6 steps
Success: 5/5 (100%)
Avg: 16.6 steps, 0.0004s
```

Figure 4: n-Queens n=8: Example solution from Trial 1. All 5 trials succeeded.

```
n-Queens n=25 - Minimum Conflicts

Trial 3 Solution (42 steps):
Q . . . . . . . . . . . . . . . . . . . . . . . .
. . . . . . . . . . . . . . Q . . . . . . . . . .
. . . . . . . . . . . . . . . . . . . . . . Q . . .
. . . . . . . . . . . . . . . . Q . . . . . . . .
. . . . . . . . . . . . Q . . . . . . . . . . . .
. . . . . . . . . . . . . . . . . . . . . . . Q . .
. . . . . . . . Q . . . . . . . . . . . . . . . .
. . . . . . . . . . . . . . . . . . . . . . . . Q .
. . . . Q . . . . . . . . . . . . . . . . . . . .
. . . . . . . . . . . . . . Q . . . . . . . . . .
[... 15 more rows ...]

Trials: 48, 59, 42, 56, 98 steps
Success: 5/5 (100%), Avg: 60.6 steps
Linear scaling: 3x board = 3.7x steps
```

Figure 5: n-Queens n=25: Example solution (first 10 rows shown). All 5 trials succeeded.

```
PSO: Rastrigin (10D)
Global min: f(0,...,0) = 0

Config 1 (swarm=30, w=0.7):
  Best: 78.97
  Avg:  80.93 ± 2.51

Config 2 (swarm=50, w=0.5):
  Best: 56.26
  Avg:  59.91 ± 2.62  <-- BEST

Config 3 (swarm=40, w=0.9):
  Best: 53.47
  Avg:  72.05 ± 13.26
```

Figure 6: PSO on Rastrigin: Config 2 best (59.91 avg). Config 3 high variance shows instability.

```
PSO: Rosenbrock (10D)
Global min: f(1,...,1) = 0

Config 1 (swarm=30, w=0.7):
  Best: 1818.54
  Avg:  2002.66 ± 141.40

Config 2 (swarm=50, w=0.5):
  Best: 568.39
  Avg:  645.59 ± 97.99  <-- BEST

Config 3 (swarm=40, w=0.9):
  Best: 2116.54
  Avg:  6016.96 ± 3759.86
```

Figure 7: PSO on Rosenbrock: Config 2 best (645.59 avg). Config 3 high inertia causes poor performance.

```
PSO Sudoku (swarm=150, iter=3000)

Starting Puzzle:
5 3 . | . 7 . | . . .
6 . . | 1 9 5 | . . .
. 9 8 | . . . | . 6 .
---------------------
8 . . | . 6 . | . . 3
4 . . | 8 . 3 | . . 1
7 . . | . 2 . | . . 6
---------------------
. 6 . | . . . | 2 8 .
. . . | 4 1 9 | . . 5
. . . | . 8 . | . 7 9

Trial 1: 13 violations, 30.70s
Trial 2: 13 violations, 31.29s
Trial 3:  6 violations, 31.36s

Summary:
  Solved: 0/3 (0%)
  Best: 6 violations
  Avg: 10.67 violations
  Avg time: 31.12s
```

Figure 8: PSO Sudoku: 0/3 solved, best 6 violations. Shows PSO struggles with discrete CSPs.

References

Atyabi, A. 2025a. Constraint Satisfaction Problems. Lecture 5, CS 4820/5820: Artificial Intelligence, University of Colorado Colorado Springs. Fall 2025.

Atyabi, A. 2025b. Search Optimization Part I-III. Lecture 7, CS 4820/5820: Artificial Intelligence, University of Colorado Colorado Springs. Fall 2025.

Russell, S. J.; and Norvig, P. 2020. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition.